GitHub User Activity CLI

Code comparison, merge decisions and a line-by-line explanation of `github_activity.py`

File	github_activity.py (183 lines)
Language	Python 3.9 or newer, built-in modules only
Purpose	Fetches the recent public activity of a GitHub user and shows it in the terminal
Project	roadmap.sh - GitHub User Activity

Contents

1. Comparing the two versions 2

1.1 What was kept from your code . . . 2
1.2 What was fixed or improved . . . 2
1.3 What was taken from, or dropped from, my first version . . . 3

2. How the program works 3

2.1 What GitHub sends back . . . 3

3. Line-by-line explanation 5

3.1 The module docstring (lines 1-5) . . . 5
3.2 Imports (lines 7-16) . . . 5
3.3 Constants (lines 18-19) . . . 5
3.4 is_valid_username() (lines 22-25) . . . 6
3.5 fetch_github_activity(): building the request (lines 28-38) . . . 6
3.6 fetch_github_activity(): sending it and handling failures (lines 40-62) . . . 7
3.7 fetch_github_activity(): checking the result (lines 64-67) . . . 8
3.8 plural() (lines 70-72) . . . 8
3.9 format_time() (lines 75-81) . . . 8
3.10 describe_event(): getting ready (lines 84-89) . . . 9
3.11 describe_event(): push events (lines 91-97) . . . 9
3.12 describe_event(): issues and comments (lines 99-106) . . . 10
3.13 describe_event(): pull requests and reviews (lines 108-116) . . . 10
3.14 describe_event(): commits, stars and forks (lines 118-125) . . . 11
3.15 describe_event(): creating and deleting (lines 127-135) . . . 11
3.16 describe_event(): releases, members, wiki and the rest (lines 137-152) . . . 11
3.17 main(): reading the username (lines 155-162) . . . 12
3.18 main(): checking, fetching and the empty case (lines 164-172) . . . 12
3.19 main(): printing the result (lines 174-179) . . . 13
3.20 Starting the program (lines 182-183) . . . 13

4. Following a run through the code 14

5. Python and web ideas used in this file 15

6. Mistakes the program handles 16

7. Practice ideas 16

1. Comparing the two versions

Both versions do the same job and follow the same plan: check the username, ask the GitHub API for the events, turn each event into a sentence, and print the list. Your version had the better structure and the richer sentences. My first version had the safer error handling. The final file keeps your design and your details, and adds the fixes below, so there is one version to submit.

1.1 What was kept from your code

Kept	Why it is good
The `/events/public` address	It returns only public events, which is exactly what this tool should show. My version used `/events`.
Function names and layout	`is_valid_username`, `fetch_github_activity`, `plural`, `describe_event`, `main` are clear and match what each one does.
Issue and pull request numbers	Sentences like `Opened issue #7` say more than a plain description.
Branch, tag and release names	`Created branch dev` and `Published release v1.0` are more useful than a general message.
`plural(count, singular, plural_form)`	It works for any word, including words that do not just add an s.
Time of each event	Seeing when something happened is useful. It is kept, with a cleaner format.
Constants with `Final`, `cast`, type hints	Your style of typing the code is kept throughout.

1.2 What was fixed or improved

Items marked Bug would give a wrong result or a crash. Improvements and Style items make the code safer, clearer or tidier.

Area	Type	Your version	Final version
Imports	Style	`from typing import Any, Final` appears twice.	One import line, plus `datetime` for time formatting.
Username check	Bug	`re.match` with `$` accepts a name that ends with a new line (`abc` plus Enter) and names that start or end with a hyphen (`-a`, `a-`).	`re.fullmatch` with a pattern that must start and end with a letter or number, 39 characters at most.
Slow replies	Bug	Only a timeout while connecting is caught. A timeout while reading the reply raises `TimeoutError` and the program crashes with a long error.	Both kinds of timeout are caught and explained.
Broken replies	Bug	Text that cannot be decoded raises `UnicodeDecodeError`, which is not caught.	`except ValueError` covers bad JSON and bad text.
Fork events	Bug	Reads `payload['forked']`. GitHub calls it `forkee`, so the result was always `Unknown`.	Reads `forkee`, so the new repository name shows.
Star events	Bug	Shows `Started watching repo`. A star on GitHub arrives as a `WatchEvent`.	Shows `Starred repo`, like the example in the project.
Push events	Bug	Counts the `commits` list. GitHub puts at most 20 commits in it, so big pushes are under-counted, and a push with no list says `with no commits`.	Uses the `size` number first, the list as a backup, and plain `Pushed to repo` if neither exists.
More events	Improvement	Issue comments, review comments, commit comments and wiki edits appeared as raw names such as `IssueCommentEvent in repo`.	Each has a readable sentence. Comments on pull requests are named correctly.
Unknown events	Improvement	`SponsorshipEvent in repo`	`Sponsorship activity in repo`

Area	Type	Your version	Final version
Missing action	Improvement	`action.capitalize()` on an empty action leaves a space at the start of the sentence.	A default word such as `Updated` is used.
Connection errors	Improvement	Prints the raw reason, for example `URL Error: [Errno -2] Name or service not known`.	A friendly sentence with the reason in brackets.
Status check	Improvement	`if response.status == 200 ... else` : `urlopen` already raises `HTTPError` for error statuses, so the `else` part almost never runs.	Removed. One line less to read and test.
Time display	Improvement	Shows `2026-10-24T10:30:00Z`.	`format_time` shows `2026-10-24 10:30 UTC`.
Input	Improvement	Spaces around the name are not removed.	`.strip()` is used, and the usage message has an example.
Request headers	Improvement	Only `User-Agent` is sent.	`Accept` is added, as the GitHub documentation recommends.
Return type	Improvement	The function returns `Any`.	`list[dict[str, Any]]` tells the reader exactly what comes back.
Style	Style	Three functions without a docstring, spaces at line ends, one blank line before `main`, `=='__main__'` without spaces.	Docstrings added and spacing tidied (PEP 8).
No activity	Style	`sys.exit(0)`	`return`, which does the same with less.

1.3 What was taken from, or dropped from, my first version

Taken	Dropped
The `Accept` header, the friendlier connection message, the wording `Starred`, the fallback sentence for unknown events, `.strip()` on the username and the example in the usage message.	The `/events` address (yours is better), `urllib.parse.quote` (not needed once the username is checked), and my shorter event sentences without numbers.

2. How the program works

The program has one job for each function. `main()` is the only one that talks to the user. It calls the others in this order:

Function	Job
`is_valid_username`	Checks that the typed name could be a real GitHub username before anything is sent.
`fetch_github_activity`	Sends the request to the GitHub API, reads the reply and handles every kind of failure.
`plural`	Writes `1 commit` or `3 commits` correctly.
`format_time`	Changes GitHub time text into a shorter, easier form.
`describe_event`	Turns one event into one short sentence.
`main`	Reads the command line, calls the other functions and prints the result.

2.1 What GitHub sends back

The API returns JSON: a list of events. Each event is a dictionary. This is a shortened example of one event:

```
{
    "type": "PushEvent",
    "repo": { "name": "octocat/Hello-World" },
    "payload": { "size": 3, "commits": [ ... ] },
    "created_at": "2026-10-24T10:30:00Z"
}
```

Field	Meaning
`type`	What kind of event it is, such as `PushEvent`, `WatchEvent` or `IssuesEvent`.
`repo.name`	The repository the event happened in, written `owner/repository`.
`payload`	The details. They are different for each type: a push has `size` and `commits`, an issue event has `action` and `issue`.
`created_at`	When the event happened, in UTC time.

3. Line-by-line explanation

Each table below shows the lines of the file in order. The first column is the line number in `github_activity.py`, the second is the code, and the third explains what it does. Blank lines are skipped because they do nothing.

3.1 The module docstring (lines 1-5)

A docstring is a text block that describes code. The one at the top describes the whole file.

Line	Code	What it does
1	`"""GitHub User Activity CLI`	Three double quotes open a docstring. Text placed first in a file is the module docstring, and this first line gives the name of the program.
2	`Fetches the recent public activity of a GitHub user with the GitHub API`	Says what the program does: it asks the GitHub API for the recent public activity of a user and shows it in the terminal.
3	`and shows it in the terminal. Uses only Python's built-in modules.`	Says that only modules that come with Python are used, so nothing needs to be installed.
5	`Usage: python github_activity.py <username>"""`	The usage line shows how to run the program. The three double quotes at the end close the docstring. Python does not run this text. It is documentation for people.

3.2 Imports (lines 7-16)

Imports bring in ready-made tools so you do not have to write them yourself.

Line	Code	What it does
7	`from __future__ import annotations`	Keeps type hints as plain text instead of working them out when the program starts. It lets you write hints such as `list[dict[str, Any]]` and `str | None`. It must be the first import in the file.
9	`import json`	Loads `json`. It converts JSON text from GitHub into Python lists and dictionaries (`json.loads`).
10	`import re`	Loads `re`, the regular expression tool. It is used to check the shape of the username.
11	`import socket`	Loads `socket`. It provides `socket.timeout`, the kind of error that happens when a connection is too slow.
12	`import sys`	Loads `sys`. It gives `sys.argv` (the words typed on the command line) and `sys.exit()` (stop the program).
13	`import urllib.error`	Loads `urllib.error`, which holds the error types `HTTPError` and `URLError`.
14	`import urllib.request`	Loads `urllib.request`, which can build and send a web request (`Request`, `urlopen`).
15	`from datetime import datetime`	Takes only the `datetime` class from the `datetime` module. It is used to read and format the time of an event.
16	`from typing import Any, Final, cast`	Loads helpers for type hints. `Any` means any type, `Final` marks a value that should not change, and `cast` tells the editor to treat a value as a certain type.

3.3 Constants (lines 18-19)

Constants are set once and never changed. They are written in capital letters.

Line	Code	What it does
18	`API_URL: Final[str] = "https://api.github.com/users/{username}/events/public"`	`API_URL` holds the address of the API. `{username}` is a hole that is filled in later with `.format`. The ending `/events/public` asks only for public events. `Final[str]` tells editors that this text constant should not be reassigned.
19	`TIMEOUT: Final[int] = 10 # seconds to wait for GitHub before giving up`	`TIMEOUT` is how many seconds to wait for GitHub before giving up. The text after `#` is a comment: Python ignores it, and it is there for the reader.

3.4 is_valid_username() (lines 22-25)

Checks the username before any request is sent.

Line	Code	What it does
22	`def is_valid_username(username: str) -> bool:`	Defines the function. It takes text and returns `bool`, which means `True` or `False`.
23	`    """Checks a username against GitHub's rules: letters, numbers and hyphens,`	The docstring opens. It states the GitHub rules: letters, numbers and hyphens, 1 to 39 characters.
24	`    1 to 39 characters, and it cannot start or end with a hyphen."""`	The docstring continues and closes. A name cannot start or end with a hyphen.
25	`    return re.fullmatch(r"[A-Za-z0-9](?:[A-Za-z0-9-]{0,37}[A-Za-z0-9])?", username) is not None`	`re.fullmatch(pattern, username)` gives a match when the WHOLE text fits the pattern, and `None` when it does not. `is not None` turns that into True or False. The pattern: `[A-Za-z0-9]` the first character must be a letter or number; `(?:...)?` an optional group; `[A-Za-z0-9-]{0,37}` up to 37 letters, numbers or hyphens; `[A-Za-z0-9]` the last character must be a letter or number. That gives 39 characters at most, and a one-letter name still works because the group is optional. The `r` before the quote makes it a raw string, so backslashes are left alone, which is good practice for patterns.

3.5 fetch_github_activity(): building the request (lines 28-38)

Gets the events from GitHub. This first part prepares the request.

Line	Code	What it does
28	`def fetch_github_activity(username: str) -> list[dict[str, Any]]:`	Defines the function. It takes the username and returns a list of dictionaries (one for each event).
29	`    """Gets the recent public events of a user from the GitHub API.`	The docstring opens and describes the job.
30	`    Stops the program with a clear message if anything goes wrong."""`	The docstring closes. It says the function stops the program with a clear message when something goes wrong.
31	`    url = API_URL.format(username=username)`	`API_URL.format(username=username)` fills the `{username}` hole. For `octocat` the result is `https://api.github.com/users/octocat/events/public`.
32	`    request = urllib.request.Request(`	Creates a request object. It is only prepared here, not sent yet. The bracket opened here closes on line 38.
33	`        url,`	The first part of the request is the address.
34	`        headers={`	`headers={` starts a dictionary of headers: extra information sent along with the request.
35	`            "Accept": "application/vnd.github+json",`	The `Accept` header tells GitHub which reply format we want. The GitHub documentation recommends this value.

Line	Code	What it does
36	`            "User-Agent": "GitHub Activity CLI", # GitHub rejects requests without one`	The `User-Agent` header names our program. GitHub rejects requests that do not have one. The text after `#` is a comment.
37	`        },`	`},` closes the headers dictionary.
38	`    )`	`)` closes `Request(`.

3.6 fetch_github_activity(): sending it and handling failures (lines 40-62)

The try block runs the risky code. The except blocks decide what to do for each kind of failure.

Line	Code	What it does
40	`    try:`	`try:` starts a block where errors are expected. If one happens, Python jumps to the matching `except` line instead of crashing.
41	`        with urllib.request.urlopen(request, timeout=TIMEOUT) as response:`	`urlopen(request, timeout=TIMEOUT)` sends the request and waits at most 10 seconds. `with ... as response` closes the connection automatically when the block ends.
42	`            data = json.loads(response.read().decode("utf-8"))`	`response.read()` gets the reply as raw bytes. `.decode("utf-8")` turns the bytes into text. `json.loads(...)` turns the JSON text into Python data, here a list of events. The result is stored in `data`.
43	`    except urllib.error.HTTPError as e:`	Runs when GitHub answers with an error status such as 404. `e` holds the details. This block must come before `URLError` because `HTTPError` is a more specific kind of `URLError`.
44	`        if e.code == 404:`	`e.code` is the status number. 404 means not found.
45	`            print(f"Error: User '{username}' not found.")`	Tells the user that the username does not exist on GitHub.
46	`        elif e.code in (403, 429):`	`elif` means otherwise, if. 403 and 429 are the statuses GitHub uses when you have made too many requests. `in (403, 429)` checks for either number.
47	`            print("Error: Rate limit exceeded. Please try again later.")`	Tells the user to wait and try again later.
48	`        else:`	`else:` covers every other error status.
49	`            print(f"HTTP Error: {e.code} - {e.reason}")`	Prints the status number and its reason text, for example `500 - Internal Server Error`.
50	`        sys.exit(1)`	Stops the program with exit code 1, which means an error. It runs after any of the messages above.
51	`    except urllib.error.URLError as e:`	Runs when the request could not reach GitHub at all: no internet, a name lookup problem or a timeout while connecting.
52	`        if isinstance(e.reason, socket.timeout):`	`e.reason` holds the underlying cause. `isinstance(e.reason, socket.timeout)` asks whether that cause was a timeout.
53	`            print("Error: Request timed out. Please try again.")`	Tells the user the request timed out.
54	`        else:`	`else:` means the cause was something else.
55	`            print(f"Error: Could not connect to GitHub ({e.reason}). Check your internet connection.")`	Shows a friendly sentence with the technical reason in brackets, and a hint to check the connection.
56	`        sys.exit(1)`	Stops the program with an error code.

Line	Code	What it does
57	`except (TimeoutError, socket.timeout):`	Catches a timeout that happens while the reply is being read. This kind is not wrapped in `URLError`. In newer Python versions `socket.timeout` and `TimeoutError` are the same thing, and listing both works on Python 3.9 too.
58	`print("Error: GitHub took too long to respond. Please try again.")`	Tells the user that GitHub was too slow.
59	`sys.exit(1)`	Stops the program with an error code.
60	`except ValueError:`	`ValueError` covers `json.JSONDecodeError` (broken JSON) and `UnicodeDecodeError` (bytes that are not valid text). Both are kinds of `ValueError`.
61	`print("Error: Failed to read the response from GitHub.")`	Tells the user that the reply could not be read.
62	`sys.exit(1)`	Stops the program with an error code.

3.7 fetch_github_activity(): checking the result (lines 64-67)

Makes sure the reply has the shape the rest of the program expects.

Line	Code	What it does
64	`if not isinstance(data, list):`	A good reply is a list of events. `isinstance(data, list)` checks that, and `not` flips the answer, so the line asks: is it NOT a list?
65	`print("Error: Unexpected data format received from GitHub API.")`	Tells the user that GitHub sent something unexpected.
66	`sys.exit(1)`	Stops the program with an error code.
67	`return cast(list[dict[str, Any]], data)`	`cast(...)` tells the editor that `data` is a list of dictionaries. It returns `data` unchanged. `return` sends the list back to `main()`.

3.8 plural() (lines 70-72)

Writes a number with the right form of a word.

Line	Code	What it does
70	`def plural(count: int, singular: str, plural_form: str) -> str:`	Defines the function. It takes a number, the singular word and the plural word, and returns text.
71	`"""Returns the singular or plural form based on the count."""`	The docstring.
72	`return f"{count} {singular}" if count == 1 else f"{count} {plural_form}"`	A conditional expression: `A if condition else B`. When `count == 1` the singular is used, otherwise the plural. The f-string joins the number and the word, giving `1 commit` or `3 commits`.

3.9 format_time() (lines 75-81)

Makes GitHub time text shorter and easier to read.

Line	Code	What it does
75	`def format_time(created_at: str) -> str:`	Defines the function. It takes the time text and returns text.
76	`"""Turns GitHub's time (2026-10-24T10:30:00Z) into a shorter form (2026-10-24 10:30 UTC)."""`	The docstring shows an example of the change.
77	`try:`	`try:` starts a block that may fail.

Line	Code	What it does
78	`        moment = datetime.strptime(created_at, "%Y-%m-%dT%H:%M:%SZ")`	`datetime.strptime(text, pattern)` reads text and turns it into a date and time object, following the pattern: `%Y` year, `%m` month, `%d` day, `%H` hour, `%M` minute, `%S` second. `T` and `Z` are letters in GitHub format (Z means UTC). The result is stored in `moment`.
79	`    except (TypeError, ValueError):`	Runs if reading failed. `ValueError` happens when the text does not match the pattern (for example `Unknown time`). `TypeError` happens when the value is not text at all.
80	`        return str(created_at)`	Returns the original value as text, so the program shows something instead of failing.
81	`    return moment.strftime("%Y-%m-%d %H:%M UTC")`	`moment.strftime(...)` does the opposite of `strptime`: it turns the date object into text in the shape we choose, such as `2026-10-24 10:30 UTC`. This line is reached only when the `try` worked.

3.10 describe_event(): getting ready (lines 84-89)

Turns one event into one sentence. The first lines pull out the parts every event has.

Line	Code	What it does
84	`def describe_event(event: dict[str, Any]) -> str:`	Defines the function. It takes one event (a dictionary) and returns text.
85	`    """Turns one GitHub event into a short sentence."""`	The docstring.
86	`    event_type = event.get("type", "UnknownEvent")`	`event.get("type", "UnknownEvent")` reads the value of the key `type`. If the key is missing, the second value is used. This is safer than `event["type"]`, which would crash on a missing key.
87	`    repo_name = event.get("repo", {}).get("name", "an unknown repository")`	Reads the `repo` dictionary (or an empty one), then its `name`, such as `octocat/Hello-World`. Chaining `.get` like this is safe even when parts are missing.
88	`    payload = event.get("payload", {})`	The payload holds the details, which differ for each type of event. An empty dictionary is used if it is missing.
89	`    action = payload.get("action", "")`	Some events have an action such as `opened` or `closed`. It is empty text when there is none.

3.11 describe_event(): push events (lines 91-97)

A push sends commits to a repository.

Line	Code	What it does
91	`    if event_type == "PushEvent":`	Runs the code below only for push events. `==` compares two values.
92	`        commits = payload.get("size")`	`payload.get("size")` is the number of commits in the push. It gives `None` if the key is missing.
93	`        if commits is None:`	`is None` checks that `size` was missing.
94	`            commits = len(payload.get("commits", []))`	Backup plan: count the items in the `commits` list with `len()`.
95	`        if commits:`	`if commits:` is True when the number is not 0 and not `None`.

Line	Code	What it does
96	`return f"Pushed {plural(commits, 'commit', 'commits')} to {repo_name}"`	Returns a sentence such as `Pushed 3 commits to octocat/Hello-World`. `plural(...)` writes `3 commits`. Single quotes are used inside the f-string because double quotes surround it.
97	`return f"Pushed to {repo_name}"`	When no commit information exists, returns a simple sentence without a number.

3.12 describe_event(): issues and comments (lines 99-106)

Issues and issue comments.

Line	Code	What it does
99	`if event_type == "IssuesEvent":`	Runs for issue events (opened, closed, reopened).
100	`issue = payload.get("issue", {})`	`issue` is the dictionary with the details of the issue.
101	`return f"{action.capitalize() or 'Updated'} issue #{issue.get('number', 'Unknown')} in {repo_name}"`	`action.capitalize()` turns `opened` into `Opened`. `or 'Updated'` is used when the action is empty, because empty text counts as False. `issue.get('number', 'Unknown')` is the issue number. Example result: `Opened issue #7 in octocat/Hello-World`.
103	`if event_type == "IssueCommentEvent":`	Runs for comments on issues.
104	`issue = payload.get("issue", {})`	Gets the issue dictionary again.
105	`kind = "pull request" if "pull_request" in issue else "issue"`	In GitHub, a pull request is also an issue, so comments on pull requests arrive as this same event type. The key `pull_request` exists only for pull requests. `"pull_request" in issue` checks whether that key exists, and `kind` becomes `pull request` or `issue`.
106	`return f"Commented on {kind} #{issue.get('number', 'Unknown')} in {repo_name}"`	Returns a sentence such as `Commented on pull request #9 in octocat/Hello-World`.

3.13 describe_event(): pull requests and reviews (lines 108-116)

Pull request events and their reviews.

Line	Code	What it does
108	`if event_type == "PullRequestEvent":`	Runs for pull request events.
109	`pull_request = payload.get("pull_request", {})`	`pull_request` is the dictionary with the details of the pull request.
110	`if action == "closed" and pull_request.get("merged", False):`	When a pull request is closed, GitHub reports the action as `closed` whether or not it was merged. This line checks for `closed` AND the `merged` flag being True. `get("merged", False)` uses False when the key is missing.
111	`action = "merged"`	Changes the action to `merged`, so the sentence says Merged.
112	`return f"{action.capitalize() or 'Updated'} pull request #{pull_request.get('number', 'Unknown')} in {repo_name}"`	Returns a sentence such as `Opened pull request #3 in octocat/Hello-World`.
114	`if event_type in ("PullRequestReviewEvent", "PullRequestReviewCommentEvent"):`	`in (...)` with two names is True if the type equals either one, so one block serves both review events.
115	`number = payload.get("pull_request", {}).get("number", "Unknown")`	Reads the number of the pull request, again with safe `.get` calls.
116	`return f"Reviewed pull request #{number} in {repo_name}"`	Returns `Reviewed pull request #3 in octocat/Hello-World`.

3.14 describe_event(): commits, stars and forks (lines 118-125)

Comments on commits, stars and forks.

Line	Code	What it does
118	`    if event_type == "CommitCommentEvent":`	Runs for comments on a commit.
119	`        return f"Commented on a commit in {repo_name}"`	Returns the sentence.
121	`    if event_type == "WatchEvent":`	Runs for watch events. GitHub creates a `WatchEvent` when someone stars a repository.
122	`        return f"Starred {repo_name}"`	Returns `Starred octocat/Hello-World`.
124	`    if event_type == "ForkEvent":`	Runs for forks.
125	`        return f"Forked {repo_name} to {payload.get('forkee', {}).get('full_name', 'Unknown')}"`	`forkee` is the name GitHub gives to the new copy of the repository. Its `full_name` is the owner and name of the copy. Example: `Forked octocat/Hello-World to you/Hello-World`.

3.15 describe_event(): creating and deleting (lines 127-135)

Creating and deleting repositories, branches and tags.

Line	Code	What it does
127	`    if event_type == "CreateEvent":`	Runs for create events.
128	`        ref_type = payload.get("ref_type", "Unknown")`	`ref_type` says what was created: `repository`, `branch` or `tag`.
129	`        if ref_type == "repository":`	If it was a repository...
130	`            return f"Created repository {repo_name}"`	...return `Created repository octocat/Hello-World`.
131	`        return f"Created {ref_type} {payload.get('ref', 'Unknown')} in {repo_name}"`	Otherwise name the branch or tag: `payload.get('ref', 'Unknown')` is its name. Example: `Created branch dev in octocat/Hello-World`.
133	`    if event_type == "DeleteEvent":`	Runs for delete events.
134	`        ref_type = payload.get("ref_type", "Unknown")`	Reads what was deleted: a branch or a tag.
135	`        return f"Deleted {ref_type} {payload.get('ref', 'Unknown')} in {repo_name}"`	Returns `Deleted branch dev in octocat/Hello-World`.

3.16 describe_event(): releases, members, wiki and the rest (lines 137-152)

The remaining event types, and a safety net for types that are not listed.

Line	Code	What it does
137	`    if event_type == "ReleaseEvent":`	Runs for release events.
138	`        tag = payload.get("release", {}).get("tag_name", "Unknown")`	Reads the tag name of the release, such as `v1.0`.
139	`        return f"{action.capitalize() or 'Published'} release {tag} in {repo_name}"`	Returns `Published release v1.0 in octocat/Hello-World`. `or 'Published'` is the default when no action is present.
141	`    if event_type == "PublicEvent":`	Runs for repositories that became public.
142	`        return f"Made {repo_name} public"`	Returns `Made octocat/Hello-World public`.
144	`    if event_type == "MemberEvent":`	Runs when a collaborator is added to a repository.

Line	Code	What it does
145	`member = payload.get("member", {}).get("login", "Unknown")`	Reads the username of that member.
146	`return f"{action.capitalize() or 'Updated'} member {member} in {repo_name}"`	Returns `Added member sam in octocat/Hello-World`.
148	`if event_type == "GollumEvent":`	Runs for wiki edits. GitHub calls its wiki system Gollum.
149	`return f"Edited the wiki of {repo_name}"`	Returns `Edited the wiki of octocat/Hello-World`.
151	`# Any other event type: show its name instead of failing`	A comment. Python ignores it. It explains the line below.
152	`return f"{event_type.removesuffix('Event')} activity in {repo_name}"`	The safety net, reached when none of the types above matched. `event_type.removesuffix('Event')` removes the word Event from the end (Python 3.9 or newer), so `SponsorshipEvent` becomes `Sponsorship` and the result is `Sponsorship activity in octocat/Hello-World`. New event types GitHub adds later will still show something readable.

3.17 main(): reading the username (lines 155-162)

main() is the control centre of the program.

Line	Code	What it does
155	`def main() -> None:`	Defines `main`. It takes no input and returns nothing.
156	`"""Reads the username from the command line and shows the activity."""`	The docstring.
157	`if len(sys.argv) != 2:`	`sys.argv` is the list of words typed on the command line. Item 0 is the script name, so a correct run has exactly 2 items. `len()` counts the items, and `!= 2` means the wrong amount: none or too many.
158	`print("Usage: python github_activity.py <username>")`	Prints the correct way to use the program.
159	`print("Example: python github_activity.py octocat")`	Prints an example.
160	`sys.exit(1)`	Stops with an error code.
162	`username = sys.argv[1].strip()`	`sys.argv[1]` is the username that was typed. `.strip()` removes spaces from both ends.

3.18 main(): checking, fetching and the empty case (lines 164-172)

Calls the other functions in order.

Line	Code	What it does
164	`if not is_valid_username(username):`	If the username does not pass the check...
165	`print(f"Error: '{username}' is not a valid GitHub username.")`	...tell the user which name was rejected...
166	`sys.exit(1)`	...and stop.
168	`events = fetch_github_activity(username)`	Calls `fetch_github_activity`. If anything fails, that function stops the program itself, so when it returns, `events` is a good list.
170	`if not events:`	`not events` is True when the list is empty.
171	`print(f"No recent public activity found for user '{username}'.")`	Tells the user there is nothing to show.

Line	Code	What it does
172	`        return`	`return` ends `main()`, so the program finishes normally.

3.19 main(): printing the result (lines 174-179)

Prints a heading and one line for each event.

Line	Code	What it does
174	`    print(f"Recent public activity for user '{username}':\n")`	Prints the heading. The `\n` inside the text is a new line, so a blank line follows the heading.
176	`    for event in events:`	`for event in events:` is a loop that takes each event in turn.
177	`        description = describe_event(event)`	Turns the event into a sentence using `describe_event`.
178	`        created_at = format_time(event.get("created_at", "Unknown time"))`	Reads the `created_at` time (or the text `Unknown time`) and makes it shorter with `format_time`.
179	`        print(f"- {description} at {created_at}")`	Prints one line for the event: a dash, the sentence, then the time. Example: `- Pushed 3 commits to octocat/Hello-World at 2026-10-24 10:30 UTC`.

3.20 Starting the program (lines 182-183)

These last two lines start everything.

Line	Code	What it does
182	`if __name__ == "__main__":`	`__name__` is a built-in variable. It equals `"__main__"` when the file is run directly with `python github_activity.py`. If another file imports this one, the name is different, so the program does not start by itself.
183	`    main()`	Calls `main()` and the program begins.

4. Following a run through the code

4.1 python github_activity.py octocat

Step	What happens	Lines
1	Python reads the file and stores every function. It reaches the last two lines, `__name__` is `"__main__"`, so `main()` is called.	182-183
2	`sys.argv` is `['github_activity.py', 'octocat']`. It has 2 items, so the usage message is skipped. `username` becomes `octocat`.	157, 162
3	`is_valid_username` checks the name against the pattern. It passes.	164, 25
4	`fetch_github_activity` builds the address `https://api.github.com/users/octocat/events/public` and a request with its headers.	31-38
5	`urlopen` sends the request. The reply is read, decoded to text and turned into a Python list.	41-42
6	The list check passes and the list is returned to `main()`.	64-67
7	If the list is empty, a message is printed and the program ends. Otherwise the heading is printed.	170-174
8	For each event, `describe_event` writes a sentence, `format_time` shortens the time, and one line is printed.	176-179

4.2 python github_activity.py no-such-user-xyz

Step	What happens	Lines
1	The username passes the check, because the name looks valid. The request is sent.	164, 41
2	GitHub answers with status 404. `urlopen` raises an `HTTPError`.	43
3	`e.code == 404`, so `Error: User 'no-such-user-xyz' not found.` is printed.	44-45
4	`sys.exit(1)` stops the program with an error code.	50

4.3 One event, from data to a printed line

Step	What happens	Lines
1	The event has `type` `PushEvent`, `repo.name` `octocat/Hello-World`, `payload.size` 3.	86-88
2	The push branch runs. `commits` is 3, which is not 0, so the sentence is built.	91-96
3	`plural(3, 'commit', 'commits')` gives `3 commits`. Result: `Pushed 3 commits to octocat/Hello-World`.	96, 72
4	`created_at` is `2026-10-24T10:30:00Z`. `format_time` changes it to `2026-10-24 10:30 UTC`.	178, 78, 81
5	The line printed is `- Pushed 3 commits to octocat/Hello-World at 2026-10-24 10:30 UTC`.	179

5. Python and web ideas used in this file

Idea	Simple meaning	Example from the file
API	A way for programs to ask another service for data	The GitHub events address
HTTP status code	A number the server sends back: 200 worked, 404 not found, 403 or 429 too many requests	`e.code == 404`
Header	Extra information sent with a request	`User-Agent`, `Accept`
JSON	A text format for lists and dictionaries that most APIs use	`json.loads(...)`
Regular expression	A pattern that describes the shape of text	`[A-Za-z0-9]`
Function (`def`)	A named block of code you can run again	`def plural(...)`
`return`	Sends a value back and ends the function	`return f"..."`
Type hint	A label for the expected type. Helps editors and is ignored when the program runs	`username: str`, `-> bool`
Docstring	Text in triple quotes that describes a function or file	Lines 23-24
Dictionary	Pairs of keys and values	`event`, `payload`
`.get(key, default)`	Reads a key and uses the default if it is missing, instead of crashing	`event.get("type", "UnknownEvent")`
List and `for` loop	A group of items, and code that runs once for each item	`for event in events:`
`if` / `elif` / `else`	Chooses which code to run	`if e.code == 404:`
`try` / `except`	Runs risky code and handles the error instead of crashing	Lines 40-62
`with`	Opens something and closes it automatically	`with urllib.request.urlopen(...)`
f-string	Text with `{}` holes that are filled with values	`f"Starred {repo_name}"`
Conditional expression	A one-line if and else that gives a value	`A if count == 1 else B`
`or` for a default	Empty text counts as False, so `or` picks the backup value	`action.capitalize() or 'Updated'`
`in`	Checks if a value is in a group, or a key is in a dictionary	`e.code in (403, 429)`
`None`	The value that means nothing	`re.fullmatch(...) is not None`
`sys.argv`	The words typed on the command line	`sys.argv[1]`
Exit code	A number a program gives when it stops: 0 is success, other numbers are errors	`sys.exit(1)`
Constant	A value set once, written in capitals	`API_URL`, `TIMEOUT`
`__name__`	Tells whether the file is run directly or imported	`if __name__ == "__main__":`

6. Mistakes the program handles

What goes wrong	What the program does	Lines
No username, or too many words	Shows the usage and an example, then stops.	157-160
The username has invalid characters	Prints that the name is not valid, then stops. Nothing is sent to GitHub.	164-166
The user does not exist (404)	Prints `User ... not found`, then stops.	44-45
Too many requests (403 or 429)	Prints that the rate limit was exceeded, then stops.	46-47
Any other error status	Prints the status number and reason, then stops.	48-49
No internet or connection failure	Prints a friendly message with the reason, then stops.	51-56
Timeout while connecting or while reading	Prints that the request timed out or GitHub was too slow, then stops.	52-53, 57-59
Broken JSON or unreadable text	Prints that the reply could not be read, then stops.	60-62
Reply that is not a list	Prints that the format is unexpected, then stops.	64-66
The user has no recent public activity	Prints a message and ends normally.	170-172
Missing fields inside an event	`.get` with default values means the program shows `Unknown` instead of crashing.	86-89
A time that cannot be read	The original text is shown.	79-80

GitHub allows only a limited number of requests per hour when you are not logged in. If you see the rate limit message, wait a while and try again.

7. Practice ideas

Try these changes one at a time, and run the program after each:

Challenge	Hint
Show only the first 5 events.	Slice the list: `events[:5]`.
Add a filter, for example `python github_activity.py octocat PushEvent`.	Allow 3 words in `sys.argv` and keep only events whose `type` matches.
Count how many events of each type there are.	Use a dictionary and add one for each `type`.
Raise the rate limit with a GitHub token.	Read it from an environment variable with `os.environ` and add an `Authorization` header.
Rebuild one function from memory.	Close this document and write `plural` or `format_time` without looking, then compare.